

Android IR Remote Modules and Sample Projects

Android-only download guide | Verified 1 August 2026

BEST FIRST DOWNLOAD

Start with **ZoserLock/android-ir-remote** to understand Android IR transmission. Use **SupIR** or **android-ir-blaster** when you need a fuller application reference. Only use AnyMote or Harmony code if your project must connect to those hubs.

What your original keywords mean

Keyword	Android development status	What to use
Peel Smart Remote	Commercial/discontinued app. No official reusable source code or public general-purpose SDK was identified.	Use SupIR or IRRemote as an open-source replacement/reference.
Yat3D TV Remote Control	No clearly identifiable Android project exists under this exact name. It may mean Yatse, which is a Kodi remote, not a universal IR SDK.	Do not design around this name until you have the exact product URL.
AnyMote Universal Smart IR	The consumer app is old, but an AnyMote Home SDK/sample repository is public. It is hub-specific and legacy.	Use only to study/control AnyMote Home hardware.
Smart IR Universal TV Remote	A generic description used by multiple apps, not one uniquely identifiable SDK.	Use ConsumerIrManager samples and an open IR code database.
Proto by Logitech	No Logitech remote product was identified with this exact name. It likely means Logitech Harmony or Philips Pronto.	For Logitech Harmony Hub, use the unofficial Android sample below.

Important: downloading an APK gives you an installable app, not reusable source code. For development, download or clone the GitHub repository.

Recommended Android downloads

1. Android IR Remote sample - best learning template

Recommended first

A small MIT-licensed Android project demonstrating ConsumerIrManager plus NEC and Sony SIRC message generation. Best when you want understandable code rather than a large finished app.

[Source repository](#)

[Download source ZIP](#)

2. SupIR - full universal IR app reference

Best full native Android reference

An open-source Android universal IR remote that uses the IRDB database and requires a phone/tablet with a built-in IR emitter. Signed APKs are offered on its GitHub Releases page. GPL-3.0 applies if you reuse its code.

[Source repository](#)

[APK releases](#)

3. Android IR Blaster - broad hardware and format support

Best feature-rich reference

A GPL-3.0 Flutter/Android application supporting built-in IR, selected USB IR dongles and audio adapters. It can import Flipper Zero, IRPLUS and LIRC files and supports learning with compatible USB hardware.

[Source repository](#)

[F-Droid app download](#)

4. IRRemote - straightforward older complete app

Useful secondary reference

A free Android app for built-in IR transmitters, forked from the older Twinone Universal TV Remote. Useful for studying a conventional customizable remote layout. Check its license before copying code.

[Source repository](#)

[F-Droid app page](#)

Hardware-specific and legacy references

AnyMote Home Android SDK and examples

Use only when connecting to an AnyMote Home hub or studying its protocol. The repository contains an Android library/sample, but it is old and should not be treated as a modern production dependency.

- [Open repository](#) | [Download source ZIP](#)

Logitech Harmony Android sample

An unofficial Android sample for authentication and control of a Harmony Hub. It is a reference, not an official supported Logitech SDK. Avoid making a new commercial product depend on Harmony cloud/service availability.

- [Open repository](#) | [Download source ZIP](#)

Google TV Anymote sample

This controls older Google TV devices over the network. Despite the name 'Anymote', it is different from the AnyMote IR hub. The protocol and devices are legacy.

- [Open repository](#) | [Download source ZIP](#)

Useful IR code database

IRDB is a community collection of infrared codes. It is data, not an Android SDK. Use it to populate brand/model command sets after checking its licensing and individual data provenance.

- [IRDB repository](#)

Official Android API

Android exposes consumer infrared from API level 19 through **android.hardware.ConsumerIrManager**. Your app must check that the device actually has an emitter before showing IR controls.

- [ConsumerIrManager API reference](#)

```
val ir = getSystemService(ConsumerIrManager::class.java) if (ir?.hasIrEmitter() == true) { val
carrierHz = 38_000 val patternUs = intArrayOf(9000, 4500, 560, 560 /* ... */)
ir.transmit(carrierHz, patternUs) } else { // Hide IR controls or connect to an external
Wi-Fi/BLE IR hub. }
```

How to download and open a template

- Install the current stable Android Studio from the official Android Developers site.
- Open a GitHub repository link. Choose **Code > Download ZIP**, or use the direct ZIP link in this guide.
- Extract the ZIP to a short local path, for example **C:\AndroidProjects\lr-remote**.
- In Android Studio choose **Open** and select the extracted project directory.
- Allow Gradle sync to finish. Old projects may require Gradle, Android Gradle Plugin, repository, namespace or SDK-level updates before they compile.
- Run first on a physical Android device. The emulator cannot reproduce a phone's infrared emitter.
- Confirm **hasIrEmitter()** returns true. If false, use a Wi-Fi/BLE/USB IR hub instead of the phone transmitter.
- [Download Android Studio](#)

Recommended implementation path

Stage	Action	Reference
1. Proof of concept	Send one known power command from a supported Android phone.	ZoserLock sample + ConsumerIrManager docs
2. Device database	Add brand/model selection and store validated commands locally.	SupIR + IRDB
3. UI and persistence	Create rooms, favorites, activities and editable button layouts.	SupIR / IRRemote
4. Learning support	Use an external IR receiver or compatible USB/Wi-Fi hub; phone IR hardware generally transmits but does not learn.	android-ir-blaster or your own IoT hub
5. Production hardening	Test timing units, carrier ranges, lifecycle, device compatibility, offline behavior and licensing.	Physical-device test matrix

Selection checklist

- Choose **built-in IR** only if your target Android devices are known to include emitters.
- Choose an **external hub** if the app must work across ordinary Android phones or must learn unknown remote commands.
- Prefer permissive licenses such as MIT/Apache-2.0 when proprietary distribution is planned. GPL projects are excellent references, but copying/deriving code may impose source-distribution obligations.
- Do not copy assets, IR databases, app names or commercial APK contents without checking their separate rights.
- Treat legacy hub integrations as prototypes because authentication and cloud endpoints may stop working.

Sources and verification notes

Links were checked on 1 August 2026. Repository activity, releases and third-party services can change. GitHub repository pages are the authoritative place to review README instructions, branches, releases and licenses before use.

- [Android ConsumerIrManager](#)
- [SupIR repository](#)
- [Android IR Blaster repository](#)
- [AnyMote SDK repository](#)
- [Logitech Harmony Android sample](#)